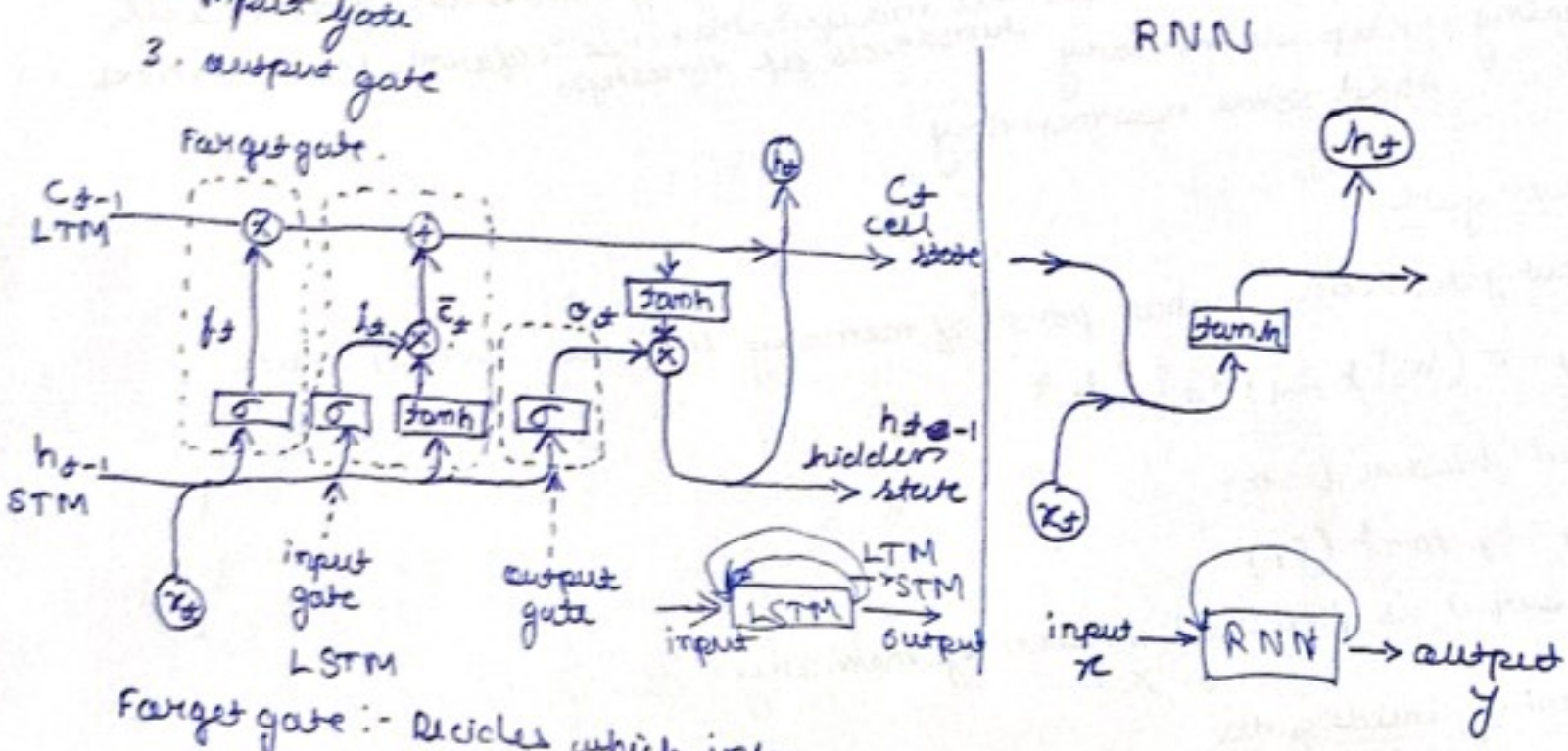
LSTM (Long short term memory)

Rehish Tiwary

- Designed to address vanishing gradient problem and capture long term dependencies in sequential data.
- LSTM do this by using specialised gates :-
  1. Forget gate
  2. Input gate
  3. output gate



Forget gate :- Decides which information discarded or forgotten.
Sigmoid activation function value : 1 means completly remember
0 means completly forget.
Removes things from Long term memory

Input gate :- Decides which new information from current input x_t should be added to cell state.
tanh activation function creates a vector of new candidate values to add to cell state
Adds things to long term memory

output state :- Decides what next hidden state should be, based on updated cell state.
Creates new short term memory which is passed to next state.

LSTM are effective in tasks like :-
Time series analysis
NLP
Sequential data problems.

Forget gate :- $f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$

where,

σ = sigmoid activation

output between 0 and 1

Update :- $f_t \cdot C_{t-1}$

Old memory can be partially erased

e.g. He grew up in france and now lives in canada. Forget gate can remove france.

Input gate

Input decision

$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$

Candidate memory

$\tilde{C}_t = \tanh(W_c[h_{t-1}, x_t] + b_c)$

Update memory

$C_t = f_t C_{t-1} + i_t \tilde{C}_t$ gradient can flow along cell state path with minimal modification. So information survives hundreds of timesteps.

Meaning: Keep old memory
add some new memory

Output gate

output gate decides what part of memory becomes hidden state.

$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$

Final hidden state:

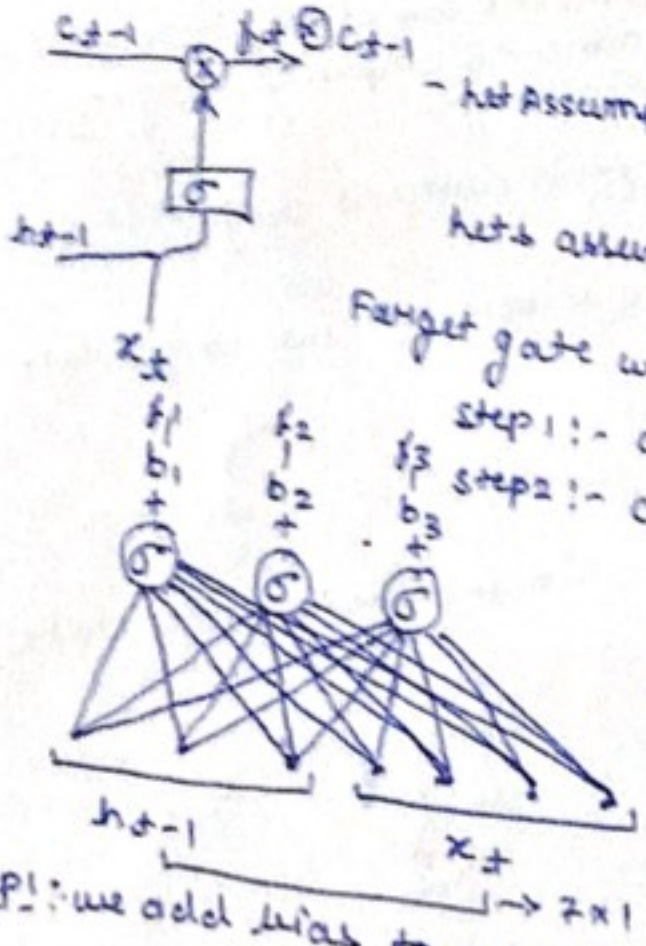
$h_t = o_t \tanh(C_t)$

So output is filtered version of memory

— x —

Working inside gates

Forget gate



- Let Assume :- No. of node = 3
$c_{t-1} = h_{t-1} =$ no. of node = 3
Lets assume input x_t = 4 dimensional vector.

Forget gate will work in 2 steps:-
step 1 :- calculate f_t
step 2 :- Calculate $f_t \otimes c_{t-1}$

Fully connected sigmoid(σ) with h_{t-1} nodes (Hidden layer) and x_t nodes.

Step 1 :- we add bias to output of sigmoid which produces f_t vectors.

Hence $f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$

W_f: 3×7, $[h_{t-1}, x_t]$: 7×1, 3×1, b_f: 3×1, 3×1

Step 2 :- $f_t \otimes C_{t-1} \rightarrow 3\times1$

~~Input gate~~

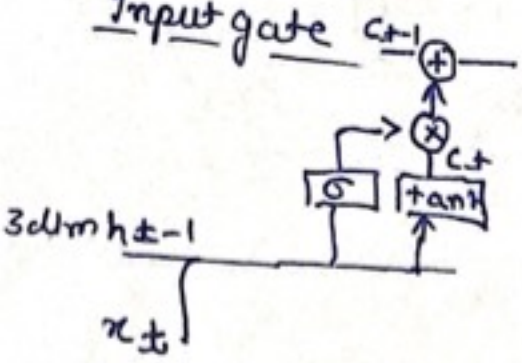
if $f_t = [0.5 \quad 0.5 \quad 0.5]$

$C_{t-1} = [4 \quad 5 \quad 6]$

$f_t \otimes C_{t-1} = [2 \quad 2.5 \quad 3]$

Hence 50% of information is removed.

Input gate

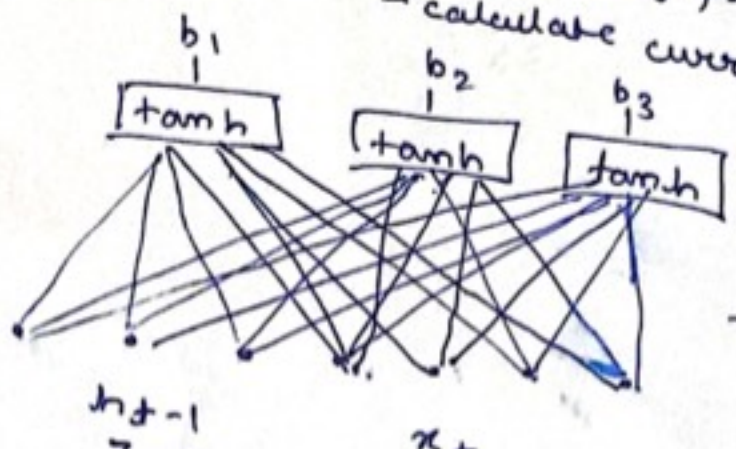


- Add some important information to cell state

3 steps :-

- Calculate candidate cell state C_{t-1}
- calculate i_t, remove information from $C_{t-1} \otimes f_t$
- calculate current cell state C_t

Step 1 :-



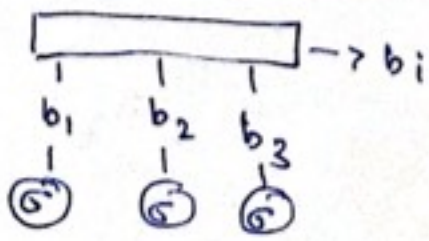
$\rightarrow W_c\ (3\times7)$

$$\underset{3\times1}{\bar{C}_t} = \tanh\Big(\underbrace{\underbrace{W_c}_{3\times7}\ \underbrace{[\underset{3\times7}{h_{t-1}}, \underset{4\times1}{x_t}]}_{7\times1}}_{3\times1} + \underbrace{b_c}_{3\times1}\Big)$$

3×1

candidate cell state :- $\tilde{C}_t$ is potential important information that we want to add to long term memory.

Step 2 :- Calculate I_t.



h_{t-1} $\quad$ x_t

$$\underset{3\times1}{i_t} = \sigma\big(W_i\,[h_{t-1}, x_t] + b_i\big)$$

Step 3 :- $\underset{3\times1}{i_t} \otimes \underset{3\times1}{\tilde{C}_t} \rightarrow \tilde{C}'_t$

Filtering information from filtering.

$$C_t = (f_t \otimes C_{t-1}) \oplus (i_t \otimes \tilde{C}_t)$$

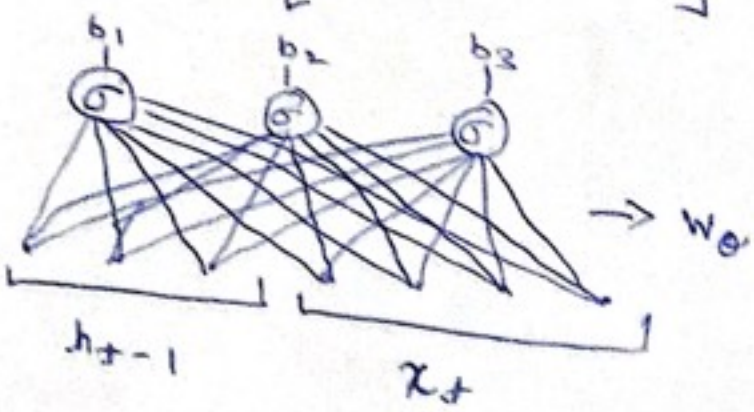
So even if information is reduced in $i_t \otimes C_t$ then $f_t \otimes C_{t-1}$ can still pass ahead, hence information is not lost in long sequences

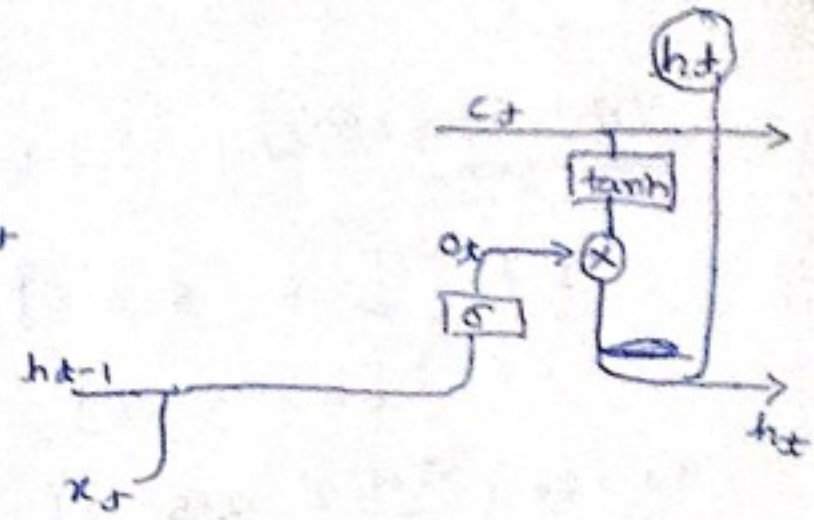
output gate

step1 :- $\tanh(C_t)$

step2 :- filteration of $\tanh(C_t) = O_t$

step3 :- $[O_t \otimes \tanh(C_t)]$





$$\underset{3\times1}{O_t} = \sigma\Big(\underbrace{\underbrace{\underset{3\times7}{W_o}\,\underbrace{[h_{t-1}, x_t]}_{3\times1}}_{3\times1} + \underbrace{b_o}_{3\times1}}_{3\times1}\Big)$$

output $\boxed{\underset{3\times1}{h_t} = \underset{3\times1}{O_t} \otimes \underset{3\times1}{\tanh(C_t)}}$